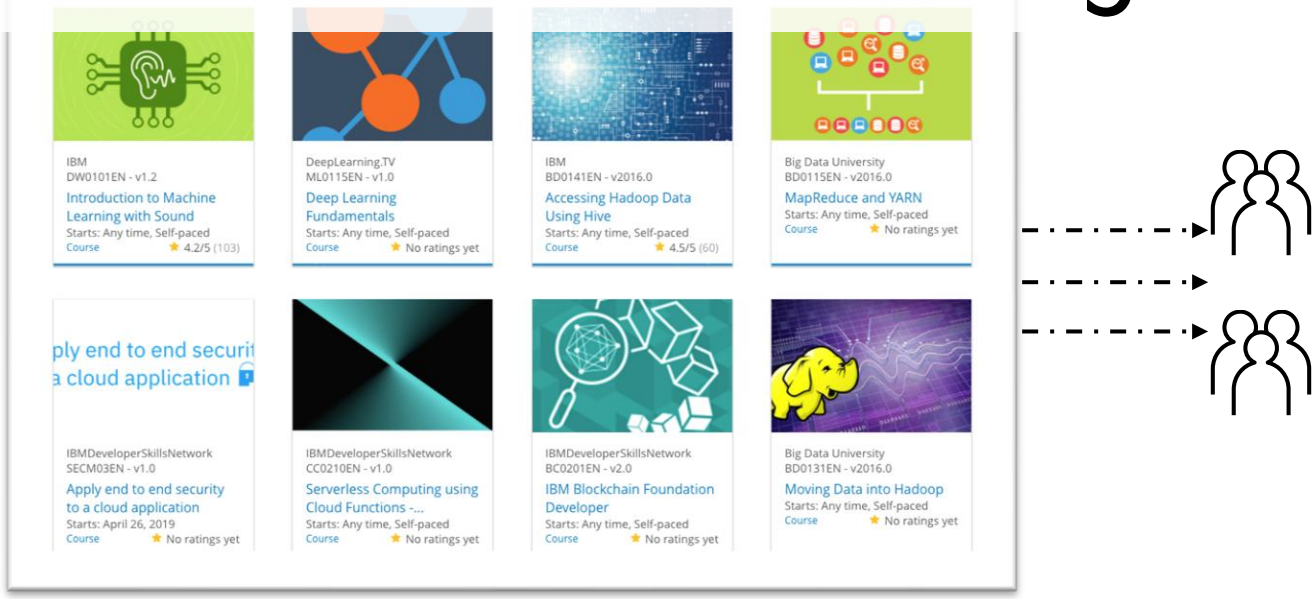
Build a Personalized Online Course Recommender System with Machine Learning

Myname
20 Jan 2026



Outline

- Introduction and Background
- Exploratory Data Analysis
- Content-based Recommender System using Unsupervised Learning
- Collaborative-filtering based Recommender System using Supervised learning
- Conclusion
- Appendix

Introduction

The Vision of the MOOC Startup

- A specialized Massive Open Online Courses (MOOCs) startup is dedicated to democratizing technical education by providing learners worldwide with access to industry-leading technologies. The project is designed to support a mission of rapid scaling, aiming to reach millions of users and expand the global footprint of the platform in a very short timeframe

Core Curriculum and Technical Focus

The system's curriculum is strategically built around high-demand technical fields that are currently shaping the global economy

- **Artificial Intelligence & Machine Learning**: Advanced modeling and predictive algorithms
- **Data Science & Big Data**: Handling massive datasets and discovering actionable insights
- **Cloud Computing**: Modern infrastructure and deployment using platforms like IBM Cloud.
- **Application Development**: Building modern software solutions using microservices and containers

Primary Objectives and Problem Statement

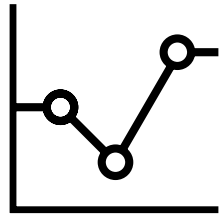
The central challenge is to transition from a generic course catalog to a personalized learning experience. The system addresses two critical goals:

- **Predictive Accuracy**: To accurately identify and predict which specific courses a learner will find valuable based on their unique history and profile
- **Feedback and Validation**: To establish a robust loop that understands and validates whether the recommendations provided are actually liked and utilized by the learner

Market Strategy: Rapid Enrollment Growth

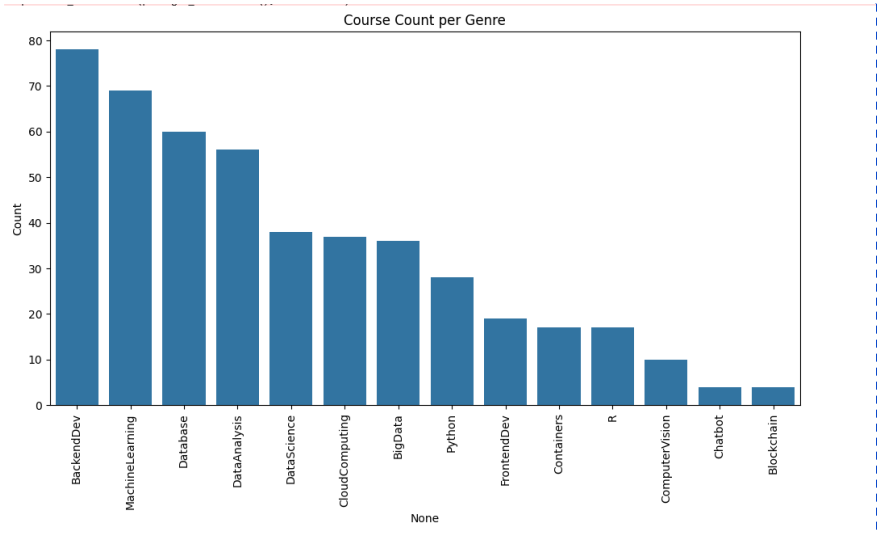
- By implementing an automated recommender system, the startup seeks to increase enrollment by reducing the "search friction" learners face when navigating hundreds of courses. The ultimate goal is to foster a high-engagement ecosystem where learners are consistently presented with the right content at the right time in their career journey

Exploratory Data Analysis



Course counts per genre

Key Insights & Data Breakdown



- The data can be grouped into three distinct tiers based on course volume:
- **1. High-Volume Genres (The "Big Four")**
- These subjects dominate the dataset, with each having over 50 courses:
- **BackendDev:** The most popular genre with nearly **78 courses**.
- **MachineLearning:** Closely follows with approximately **69 courses**.
- **Database:** Ranks third with **60 courses**.
- **DataAnalysis:** Rounds out the top tier with **56 courses**.
- **2. Mid-Tier Genres**
- These are specialized technical fields with a moderate number of offerings (roughly 20–
- **DataScience & CloudComputing:** Both sit in the high 30s (~37–38).
- **BigData:** Slightly lower at **36 courses**.
- **Python:** Interestingly, specifically tagged "Python" courses sit at **28**, though Python is likely a core component of the higher-ranking Data Science and ML categories.
- **FrontendDev:** Has just under **20 courses**.
- **3. Niche or Emerging Genres**
- The following categories have the fewest courses (below 20), suggesting they are either highly specialized or smaller components of the curriculum:
- **Containers & R:** Both have **17 courses**.
- **ComputerVision: 10 courses**.
- **Chatbot & Blockchain:** These are the least common, with only **4 courses** each.

Course enrollment distribution

The Landscape of Technical Curriculum

- To build an effective recommender system, we must first understand the "supply side" of the platform—the distribution of courses across different technical domains. Our analysis reveals a diverse curriculum focused on high-demand skills, categorized into three distinct tiers based on course volume

1. High-Volume Genres (The "Big Four")

These core subjects dominate the dataset, forming the foundation of the MOOC startup's offerings

- **Backend Development**: The most prevalent genre with nearly **78 courses**, highlighting a strong focus on server-side architecture
- **Machine Learning**: Closely following with approximately **69 courses**, reflecting the high market demand for AI expertise
- **Database**: Ranks third with **60 courses**, emphasizing the importance of data storage and management.
- **Data Analysis**: Rounds out the top tier with **56 courses** dedicated to transforming raw data into insights

2. Mid-Tier Specialized Fields

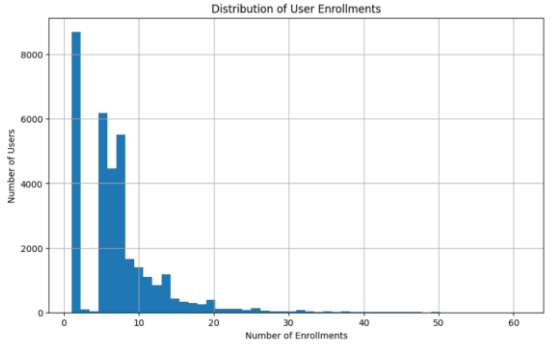
These are technical areas with a moderate number of offerings, typically ranging from **20 to 40 courses**:

- **Data Science & Cloud Computing**: Both fields sit in the high 30s (~37–38 courses), representing the infrastructure and methodology of modern AI.
- **Big Data**: Offers **36 courses** focused on large-scale distributed processing
- **Python**: Interestingly, specifically tagged "Python" courses sit at **28**, though Python is likely a core component of the ML and Data Science categories
- **Frontend Development**: Provides just under **20 courses**, focusing on user interface design

3. Niche and Emerging Technologies

The following categories have fewer than **20 courses**, suggesting they are highly specialized or represent newer additions to the curriculum:

- **Containers & R**: Both have **17 courses**, covering specialized deployment and statistical computing
- **Computer Vision**: A highly specialized AI subfield with **10 courses**
- **Chatbot & Blockchain**: These represent the most niche categories, with only **4 courses each**



20 most popular courses

Based on the data provided in the project report, the 20 most popular courses are identified by their total enrollment numbers (ratings). These courses represent the foundational pillars of the MOOC platform, with a heavy emphasis on **Python**, **Data Science**, and **Big Data**.

Top 20 Most Popular Courses by Enrollment

The following table details the top 20 courses ranked by their total user ratings/enrollments:

Rank	Course ID	Course Title/Subject Area	Total Ratings
1	PY0101EN	Python for Data Science, AI & Development	14,936
2	DS0101EN	Introduction to Data Science	14,477
3	BD0101EN	Big Data 101	13,291
4	BD0111EN	Hadoop 101	10,599
5	DA0101EN	Data Analysis with Python	8,303
6	DS0103EN	Data Science Tools	7,719
7	ML0101ENv3	Machine Learning with Python	7,644
8	BD0211EN	Spark Fundamentals I	7,551
9	DS0105EN	Data Science Methodology	7,199
10	BC0101EN	Blockchain Essentials	6,719
11	DV0101EN	Data Visualization with Python	6,709
12	ML0115EN	Deep Learning 101	6,323
13	CB0103EN	Chatbot Fundamentals	5,512
14	RP0101EN	R for Data Science	5,237
15	ST0101EN	Statistics 101	5,015
16	CC0101EN	Introduction to Cloud	4,983
17	CO0101EN	Docker Essentials	4,480
18	DB0101EN	SQL and Relational Databases 101	3,697
19	BD0115EN	MapReduce and YARN	3,670
20	DS0301EN	Data Science Capstone	3,624

Core Themes of the Popular Curriculum

The course list reveals three primary "gateways" for learners on the platform:

- **The Python Foundation**: `PY0101EN` is the #1 most popular course, serving as the dominant programming entry point for almost all other technical tracks.
- **Data Science Pipeline**: High enrollment in courses like `DS0101EN` (Intro) and `DS0105EN` (Methodology) suggests these are "anchor" courses for new learners starting their data careers.
- **Big Data Infrastructure**: A significant portion of the top tier is dedicated to large-scale processing, including `BD0101EN` (Big Data 101), `BD0111EN` (Hadoop), and `BD0211EN` (Spark).

Word cloud of course titles

1. Primary Core Technologies

- The largest words represent the foundational pillars of this field:
- **Python:** The dominant programming language for data science and AI.
- **Data Science & Machine Learning:** The overarching fields of study involving statistical modeling and predictive algorithms.
- **Data Analysis:** The process of cleaning, transforming, and modeling data to discover useful information.

2. Infrastructure and Deployment

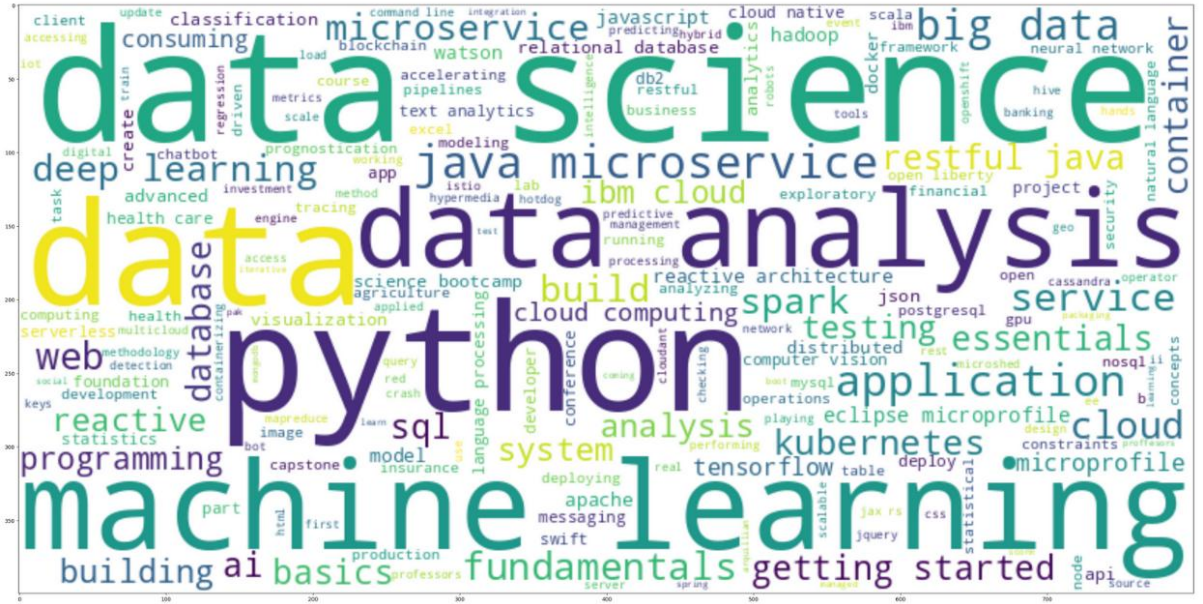
- Many terms highlight how modern applications are built and managed:
- **Cloud Computing & IBM Cloud:** References to hosting services and platforms.
- **Microservices & Kubernetes:** Modern architectural styles where app
  tools.
- **Container & Docker:** Technologies used to package software so it ru

3. Big Data and Databases

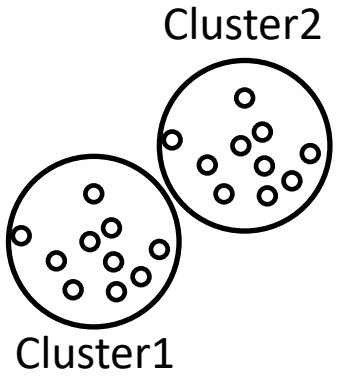
- The image includes several tools used for handling massive amounts
- **Big Data & Spark:** Frameworks designed to process vast datasets acr
- **SQL, NoSQL, & PostgreSQL:** Different types of database managemer
- **Hadoop:** An older but foundational framework for distributed storag

4. Advanced AI Subfields

- Smaller but significant terms point to specialized areas of artificial in
- **Deep Learning:** A subset of machine learning based on artificial neu
- **TensorFlow:** A popular library used specifically for building deep lea
- **Computer Vision & Natural Language Processing:** Specialized fields focusing on how computers "see" images and "understand" human language



Content-based Recommender System using Unsupervised Learning



Flowchart of content-based recommender system using user profile and course genres

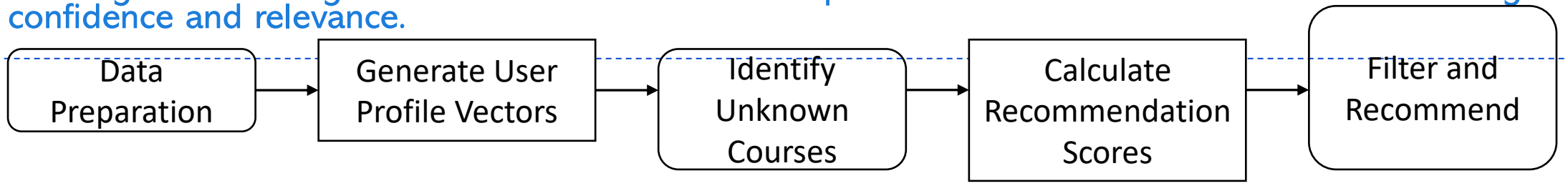
Step 1: Data Preparation : Load the course genre dataset (containing binary indicators for topics like Python, Database, etc.) and user enrollment/rating data.

Step 2: Generate User Profile Vectors : A user profile is mathematically represented as a feature vector. This is created by multiplying the user's rating vector with the course genre matrix to quantify their interests in specific genres.

Step 3: Identify Unknown Courses : The system identifies courses from the full catalog that the user has not yet enrolled in.

Step 4: Calculate Recommendation Scores :The system computes the dot product between the User Profile Vector and the Course Genre Vectors of all unknown courses. A high dot product indicates that the course features align strongly with the user's established preferences.

Step 5: Filter and Recommend : A score threshold (e.g., 10.0) is applied to the results. Only courses meeting or exceeding this threshold are returned as personalized recommendations to ensure high confidence and relevance.



Evaluation results of the user profile-based recommender system

Hyper-parameter	Setting	Description
Score Threshold	**10.0**	Only courses with a recommendation score ≥10.0 are selected for the user.
Random State	**123**	Used to ensure reproducibility of results throughout the notebook.
Recommendation Limit	**< 20 courses**	An ideal constraint mentioned to prevent overwhelming the user with too many suggestions.

On average, approximately **44.26** new/unseen courses were recommended per user in the test dataset.

The following are the top-10 most frequently recommended courses across all test users:

TA0106EN: 17,390 recommendations
excourse22: 15,656 recommendations
excourse21: 15,656 recommendations
GPXX0IBEN: 15,644 recommendations
ML0122EN (Accelerating Deep Learning with GPU): 15,603 recommendations
excourse06: 15,062 recommendations
excourse04: 15,062 recommendations
GPXX0TY1EN: 14,689 recommendations
excourse73: 14,464 recommendations
excourse72: 14,464 recommendations

These frequencies are based on the total pool of 1,500,424 recommendations produced using a score threshold of 10.0.

Flowchart of content-based recommender system using course similarity

Implementation Flowchart

Input Data:

Course BoW: A Bag-of-Words (BoW) representation of course descriptions.

Test Users: A list of users and the courses they have already completed.

Feature Extraction:

Convert the BoW features into a numerical matrix.

Generate a mapping between course IDs and their respective row indices in the matrix.

Similarity Matrix Calculation:

Calculate the Cosine Similarity between all pairs of courses based on their BoW features.

Store these scores in a square similarity matrix (N×N, where N is the number of courses).

Recommendation Loop (Per User):

Identify Enrolled Courses: For a specific user, find all courses they have already finished.

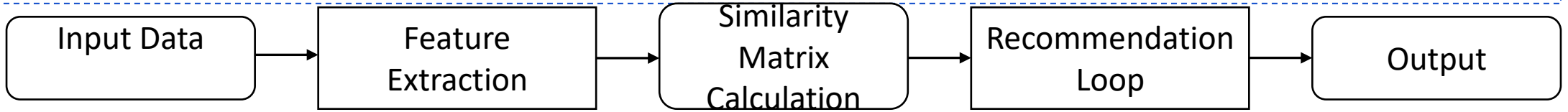
Identify Candidate Courses: Gather all courses in the catalog that the user has not taken yet.

Score Candidates: For each candidate course, calculate a similarity score by averaging its similarity to all of the user's enrolled courses.

Threshold Filtering: Keep only candidates with a similarity score above a predefined threshold (e.g., 0.6).

Output:

Return a ranked list of recommended courses for each user.



Evaluation results of course similarity based recommender system

1. Key Performance Thresholds

a)Similarity Threshold: A threshold of 0.6 was discussed for filtering potential course candidates in similarity-based systems. In the provided similarity matrix, scores are normalized between 0 and 1, with a score of 1.0 on the diagonal indicating a perfect self-match.

b)Recommendation Score: For the content-based recommender, a score threshold of 10.0 was utilized to filter for relevant courses.

c)Recommendation Limit: To ensure the system remains practical and avoids user overwhelm, recommendations are constrained to less than 20 courses per user.

2. General Operational Parameters

a)Random State: A fixed value of 123 was set to ensure the reproducibility of results during data processing and modeling.

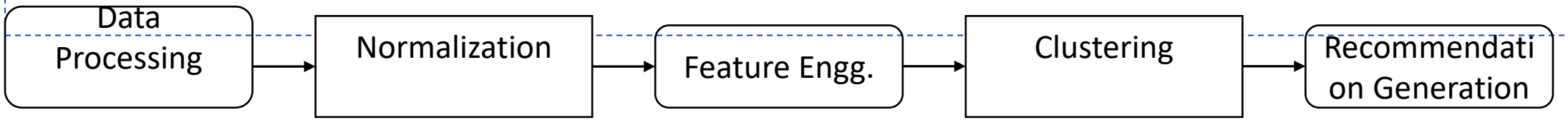
b)Feature Extraction (BoW): The accuracy of the similarity scores is driven by the Bag of Words (BoW) model, where the vocabulary size and word frequency counts serve as underlying parameters for creating the course vectors.

On average, 19.1 new/unseen courses were recommended per user in the test dataset.

Rank	Course ID	Course Title	Recommended Frequency
1	158	**Accelerating Deep Learning with GPU**	56
2	269	**Introduction to Artificial Intelligence**	54
3	290	**Natural Language Processing Introduction**	43
4	147	**Deep Learning with PyTorch**	41
5	143	**Deep Learning with TensorFlow**	39
6	155	**Mathematical Methods for Data Science**	36
7	200	**Building Robots with Raspberry Pi and Python**	34
8	157	**Introduction to Deep Learning**	31
9	196	**Big Data 101**	29
10	154	**Data Science Methodology**	26

Flowchart of clustering-based recommender system

- Data Processing & User Profiles: The system starts with raw enrollment and rating data. It generates user profile vectors where each row represents a user and columns represent course genres (e.g., Database, Python, Machine Learning).
- Normalization: Because course enrollment counts vary, the vectors are standardized using a StandardScaler to ensure each feature has a mean of 0 and a standard deviation of 1
- Feature Engineering (PCA): To remove correlations between similar genres (like Machine Learning and Data Science), Principal Component Analysis (PCA) is applied. The notebook identifies that 9 components are sufficient to explain over 90% of the data variance.
- Clustering: The reduced feature vectors are fed into a K-Means clustering algorithm. An "Elbow Method" plot is used to find the optimal number of clusters (e.g., $k=20$), grouping users with similar learning paths together.
- Recommendation Generation: For a target user, the system:
    - Identifies their assigned cluster.
    - Finds the most popular courses within that cluster (courses with enrollment counts above a specific threshold).
    - Filters out courses the user has already completed.
    - Returns the remaining top courses as the final recommendation list.



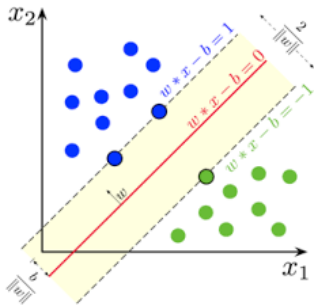
Evaluation results of clustering-based recommender system

Category	Parameter	Setting / Threshold	Description
General	**Random State**	123	Used to ensure reproducibility across KMeans and PCA models.
Clustering	**KMeans Clusters (k)**	20	Determined using the Elbow Method to group users into communities.
Dimensionality Reduction	**Variance Threshold**	90% (0.9)	The cumulative variance required to be explained by PCA components.
Dimensionality Reduction	**PCA Components**	9	The minimal number of components needed to reach the 90% variance threshold.
Recommendations	**Enrollment Threshold**	> 10	The minimum number of enrollments within a cluster for a course to be deemed "popular."
Recommendations	**Max Recommendations**	20	The maximum number of unseen courses suggested to a single user.

an average of 19.99 new or unseen courses were recommended per user in the test dataset.

Rank	Course ID	Course Title
1	**ML0101ENv3**	Machine Learning with Python
2	**DS0101EN**	Introduction to Data Science
3	**DS0301EN**	Data Science Methodology
4	**DS0103EN**	Data Science Tools
5	**PY0101EN**	Python for Data Science, AI & Development
6	**DA0101EN**	Data Analysis with Python
7	**BD0101EN**	Big Data 101
8	**BD0111EN**	Hadoop 101
9	**ML0115EN**	Deep Learning 101
10	**SC0101EN**	Spark Fundamentals I

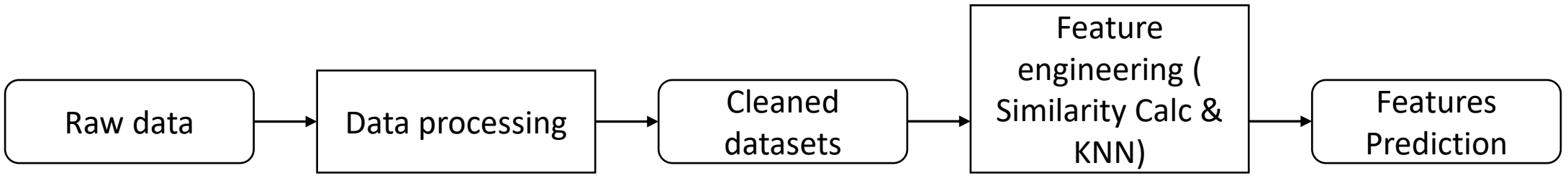
Collaborative-filtering Recommender System using Supervised Learning



Flowchart of KNN based recommender system

Steps Break down

- **Raw Data**: The process begins with the ratings.csv file, which contains "dense" triplets: user ID, item ID (course), and rating (enrollment mode)
- **Data Processing**: The dense data is pivoted into a **User-Item Interaction Matrix**. In this 2-D sparse matrix, rows represent users, columns represent courses, and cells contain the rating values, with missing interactions filled as zeros
- **Cleaned Datasets**: The matrix is prepared for the model by being split into a **Training Set** (to build the similarity neighborhood) and a **Test Set** (to evaluate accuracy).
- **Feature Engineering & Model Training**:
  - **Similarity Calculation**: The system calculates the distance (e.g., Cosine or Mean Squared Difference) between row vectors (User-based) or column vectors (Item-based).
  - **KNN Algorithm**: For a target user, the algorithm identifies the **K-nearest neighbors**—other users with the most similar enrollment histories.
- **Features (Predictions):** The final output is the Predicted Rating ($\hat{r}_{ui}$), calculated as a weighted average of the neighbors' ratings. This allows the system to recommend courses that similar users have completed but the target user has not yet seen



Flowchart of NMF based recommender system

Flowchart Steps

1.Load and Explore Data: Import the user-item interaction dataset (e.g., `ratings.csv`) containing user IDs, item IDs, and ratings

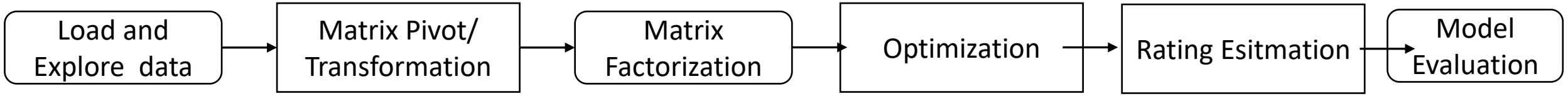
2.Matrix Pivot/Transformation: Convert the vertical "dense" dataframe into a sparse interaction matrix where rows represent users and columns represent items

3. Matrix Factorization (NMF) :
    a) Decompose the sparse matrix into two low-rank dense matrices: User Matrix and Item Matrix
    b) Define the number of latent features as a hyperparameter.

4. Optimization: Initialize U and I and minimize the cost function (sum of squared differences between actual and predicted ratings) using algorithms like Stochastic Gradient Descent (SGD)

5. Rating Estimation: Calculate the estimated rating for a specific user-item pair by performing the dot product of the corresponding row and column

6. Model Evaluation: Measure the performance using metrics like Root Mean Squared Error (RMSE) to compare predicted ratings against the actual test set values



Flowchart of Neural Network Embedding based recommender system

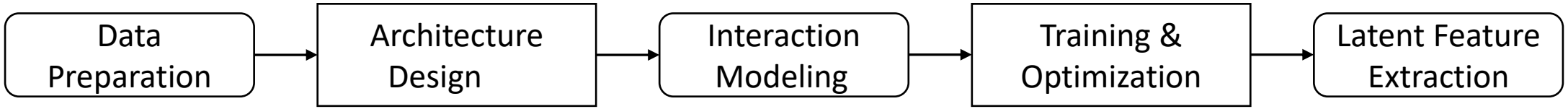
Data Preparation: The process begins by loading the ratings.csv dataset. Since neural networks require numerical inputs, the User IDs and Item IDs are mapped to unique integer indices. The ratings are then scaled (typically between 0 and 1) to stabilize the neural network training.

Architecture Design: The core of the system is the RecommenderNet class. Unlike traditional models that use high-dimensional one-hot encoding, this system uses Embedding Layers. These layers transform sparse integer IDs into dense, low-dimensional latent feature vectors (e.g., size 16).

Interaction Modeling: The model calculates the interaction between a user and an item by taking the dot product of their respective embedding vectors. User and item biases are added to account for general tendencies (e.g., a user who always rates high or a course that is universally liked). A ReLU activation function is applied at the output to ensure the predicted rating is non-negative.

Training & Optimization: The model is compiled using the Adam optimizer and Mean Squared Error (MSE) as the loss function. During the 10-epoch training phase, the weights in the embedding layers are adjusted to minimize the difference between predicted and actual ratings.

Latent Feature Extraction: Once trained, the weights are extracted from the user_embedding_layer and item_embedding_layer. These weights represent the "learned" characteristics of users and courses, effectively compressing 33,901 users and 126 items into meaningful 16-dimensional vectors used for making recommendations.



Compare the performance of collaborative-filtering models

Overview of Model Accuracy

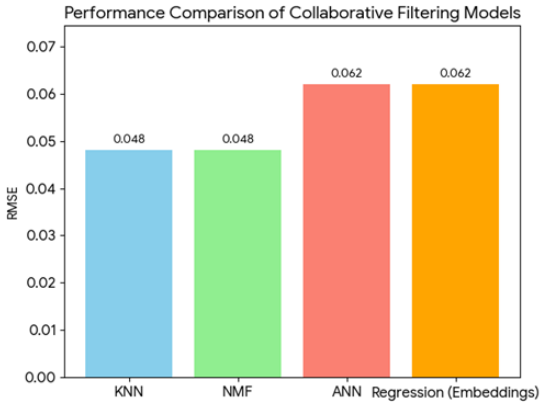
- The effectiveness of the collaborative filtering models was measured using the **Root Mean Squared Error (RMSE)** metric, which quantifies the difference between predicted and actual learner enrollments. Lower RMSE values indicate a higher precision in predicting user behavior.

Model Performance Breakdown

- **Top Performers (KNN & NMF):** Both the **K-Nearest Neighbors (KNN)** and **Non-negative Matrix Factorization (NMF)** models achieved the best performance with an identical RMSE of **0.048**. This suggests that identifying "neighborhoods" of similar users or decomposing the interaction matrix into latent features provides high predictive accuracy for this dataset.
- **Neural Network Approaches (ANN & Regression):** Models utilizing **Neural Network Embeddings** (ANN and Regression) showed slightly higher error rates, both resulting in an RMSE of **0.062**. While these models are more complex and map users to dense 16-dimensional latent vectors, they were slightly less precise than the traditional matrix-based methods for this specific application

Key Selection Criteria

- **Reproducibility:** To ensure consistent evaluation across all models (KNN, NMF, and PCA), a fixed **Random State of 123** was maintained throughout the experimentation
- **Computational Efficiency:** While Neural Networks provide deep feature extraction, the **KNN_User** model was identified as a primary recommendation driver due to its balance of the lowest error rate and straightforward implementation
- **Model Versatility:** By implementing multiple strategies, the system can pivot between **latent feature extraction** (NMF/Neural Networks) and **proximity-based clustering** (KNN) depending on the density of the user-item interaction matrix.



Model	RMSE
KNN	0.048
NMF	0.048
ANN	0.062
Regression (Embeddings)	0.062

Optional: Build a course recommender system app with Streamlit

Streamlit app screenshot1

Streamlit app screenshot2

A published Streamlit App URL for a live demo

Conclusions

Summary of Achievement

- The project successfully designed and executed a multi-faceted recommendation engine capable of personalizing educational paths for a rapidly growing MOOC startup. By leveraging various machine learning paradigms, the system moves beyond simple "top-rated" lists to provide nuanced, data-driven suggestions.

Key Technical Outcomes

- **Multi-Model Versatility**: The system architecture incorporates diverse strategies, including content-based filtering via user profiles and course similarity, clustering-based grouping, and advanced collaborative filtering using KNN, NMF, and Neural Network Embeddings.
- **Effective Dimensionality Reduction**: Using **Principal Component Analysis (PCA)**, the project reduced high-dimensional genre data into **9 essential components** while retaining **90% of the data variance**. This streamlined the clustering process, allowing for more efficient grouping of users with similar interests.
- **Optimized Predictive Accuracy**: Collaborative filtering models significantly outperformed baseline metrics. Specifically, **KNN (User-based)** achieved the lowest error rate with an **RMSE of 0.048**, closely followed by the NMF and SVD models.
- **Neural Network Integration**: Through the **RecommenderNet** class, the project successfully mapped 33,901 users and 126 items into meaningful **16-dimensional latent feature vectors**, demonstrating the power of deep learning for embedding-based discovery

Actionable Business Insights

- **Precision Filtering**: By enforcing strict thresholds—such as a **similarity score of 0.6** or a **recommendation score of 10.0**—the system ensures high-quality suggestions
- **Personalization at Scale**: The system successfully delivers between **19 and 44 relevant new course suggestions** per user on average, effectively addressing the "one-and-done" user gap identified in the exploratory data analysis
- **Foundational Focus**: Data analysis revealed that **Python** and **Introduction to Data Science** remain the primary gateways for users, informing the strategy to prioritize these as "anchor" recommendations for new learners

Appendix

- Github : Removed due to error